

Experiment 1: Linear Search (Iterative)

Aim

To implement the **Linear Search** algorithm using Python and analyze its time complexity.

Algorithm

Step 1: Start.

Step 2: Read the number of elements n.

Step 3: Read the elements of the array.

Step 4: Read the key element to be searched.

Step 5: Compare the key with each element of the array one by one.

Step 6: If the key is found, display its index (position).

Step 7: If the key is not found after checking all elements, display "**Element not found**".

Step 8: Stop.

Source Code (Python)

```
main.py    Share  Run
```

```
1  # Linear Search (Iterative)
2
3  n = int(input("Enter number of elements: "))
4
5  arr = []
6
7  print("Enter the elements:")
8  for i in range(n):
9      arr.append(int(input()))
10
11  key = int(input("Enter element to search: "))
12
13  found = False
14
15  for i in range(n):
16      if arr[i] == key:
17          print("Element found at index", i)
18          found = True
19          break
20
21  if not found:
22      print("Element not found")
```

Output

Output

```
Enter number of elements: 4
Enter the elements:
10
27
30
45
Enter element to search: 45
Element found at index 3

=== Code Execution Successful ===
```

Time Complexity

- **Best Case (Ω):** $O(1)$
- **Average Case (Θ):** $O(n)$
- **Worst Case (O):** $O(n)$

Result

The **Linear Search algorithm** was implemented successfully in Python. The program searches the elements one by one and displays the index if the element is found; otherwise, it displays "**Element not found**". This matches the objective and complexity described in the lab manual.